

Fine-tuning

How do you teach LLM to answer your specific questions?

Maryam Abdolali
KNTU, Fall 2025

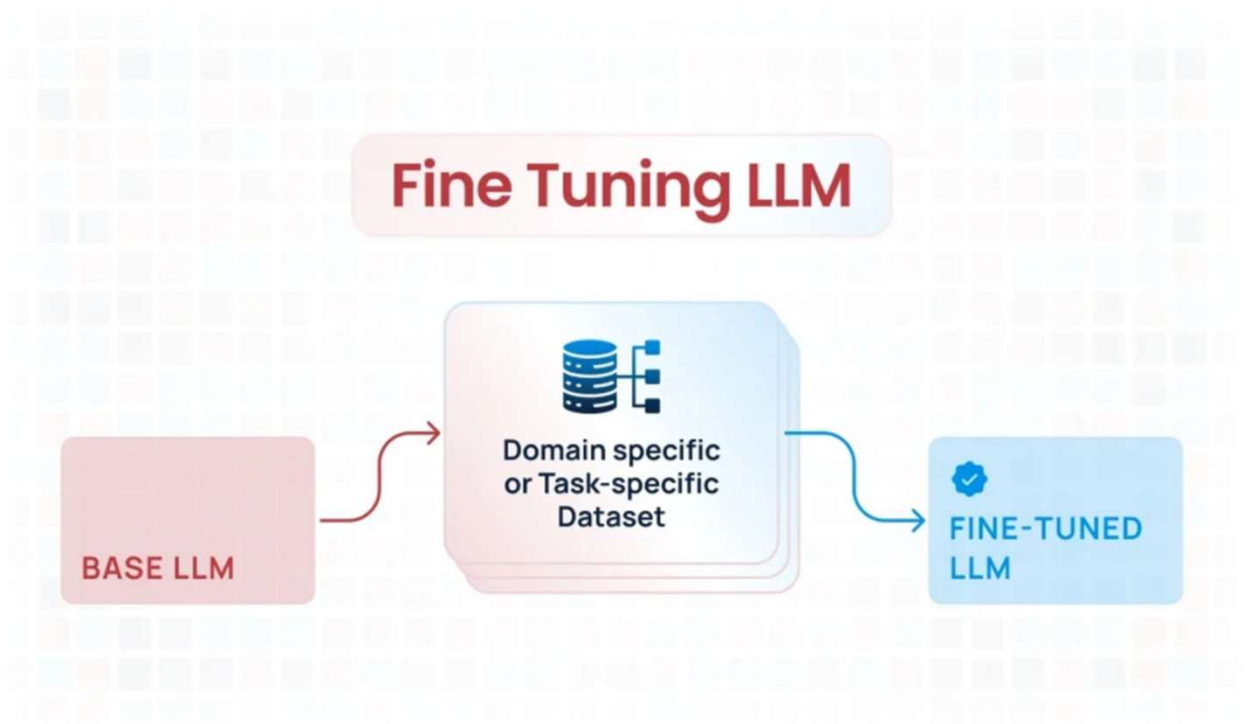
Why Fine-tuning:

When you should fine-tune:

- Need domain-specific knowledge (medical, legal, finance)
- Want specific tone/style (your brand voice)
- Require particular formats (structured outputs)
- Have proprietary data that can't go to OpenAI
- Need better performance than prompting provides

Three types of fine-tuning:

- Full fine-tuning
- Parameter efficient fine tuning
- Instruction Tuning + RLHF 



Full fine-tuning

Train all parameters of the model on your data.

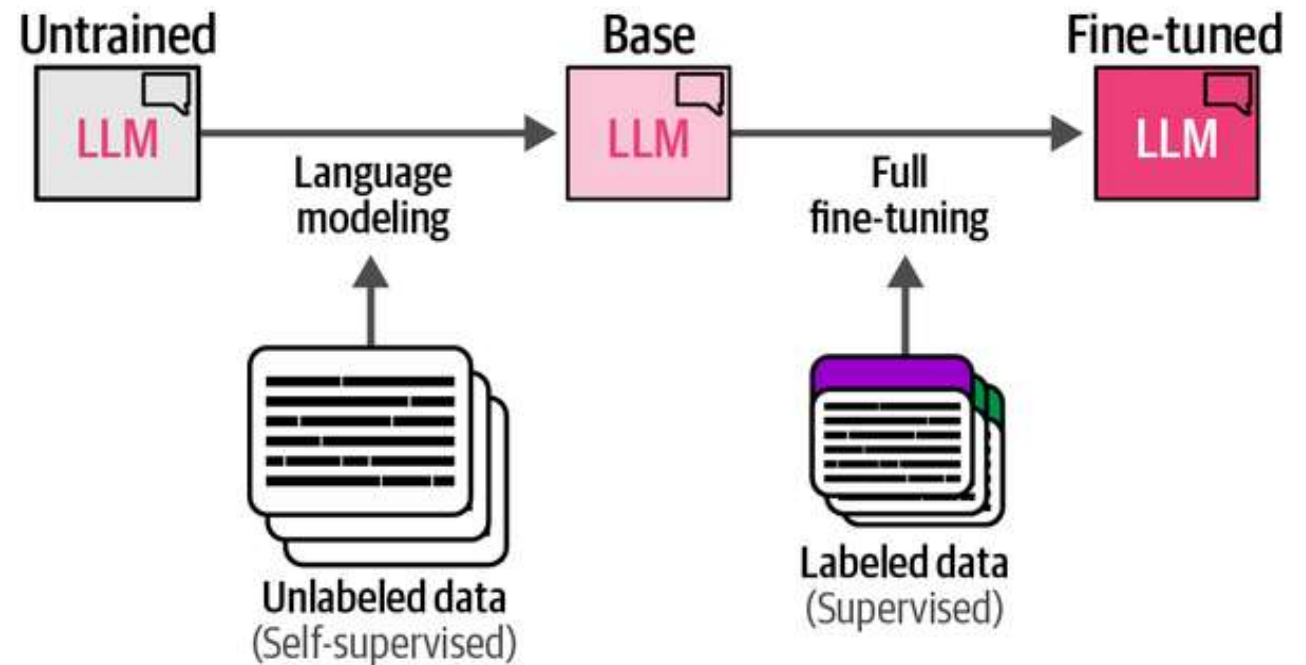
Pros:

- ✓ Maximum control and performance
- ✓ Best results for obscure, domain-specific tasks

Cons:

- ✓ Extremely resource-intensive (Meta's Llama 3.1 has 8B-405B parameters)
- ✓ Risk of catastrophic forgetting (model overwrites general knowledge)
- ✓ Risk of overfitting on small datasets
- ✓ Requires massive GPU memory

Use when: You have huge datasets, massive compute, and need the absolute best performance aka never.



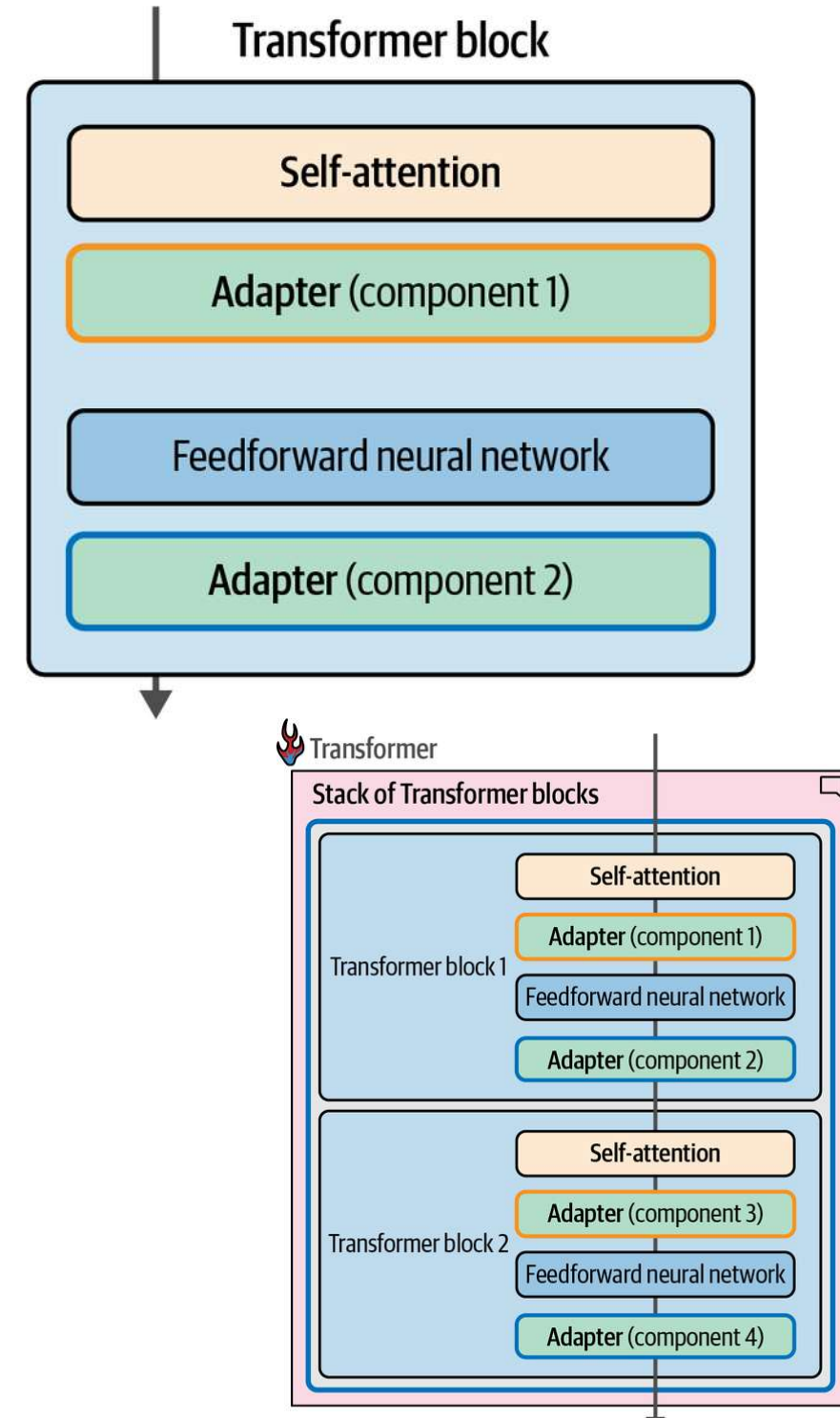
Fun fact: Did you know, training GPT-4 involved ~25,000 A100 GPUs over ~90-100 days, costing OpenAI nearly \$100 million!

Parameter-Efficient Fine-Tuning (PEFT)

- **Problem:** Updating **all parameters** of a model has a large potential of increasing its performance but comes with several disadvantages
 - costly to train,
 - has **slow training times**, and **requires significant storage**
- **Goal:** fine-tuning pretrained models at **higher computational efficiency**.
- **Approach:**
 - A transformer is mostly stacks of **linear layers** (attention projections and MLP projections). A linear layer does:
$$y = Wx \text{ (} W \text{ is pretrained)}$$
 - Full fine-tuning updates **all** of W (huge memory: gradients + optimizer states + activations).
 - **PEFT** methods instead **freeze W and learn a small add-on ΔW** :
$$y = (W + \Delta W)x$$
 - The whole game is: make ΔW expressive enough, but cheap to train/store.

Adapters

- Add a set of additional modular components inside the Transformer that can be fine-tuned to improve the model's performance on a specific task ~~without having to fine-tune all the model weights~~. This saves a lot of time and compute.
- Has been shown fine-tuning 3.6% of the parameters of BERT for a task can yield comparable performance to fine-tuning all the model's weights.
- *Adapters add a small number of weights in certain places in the network that can be fine-tuned efficiently while leaving the majority of model weights frozen.*
- Adapter 1 can be a specialist in, say, medical text classification, while Adapter 2 can specialize in named-entity recognition (NER). You can download specialized adapters from <https://oreil.ly/XraXg>.
- A lot of these earlier adapters were more focused on BERT architectures



Adapter modules

- Each adapter is a **bottleneck MLP**:

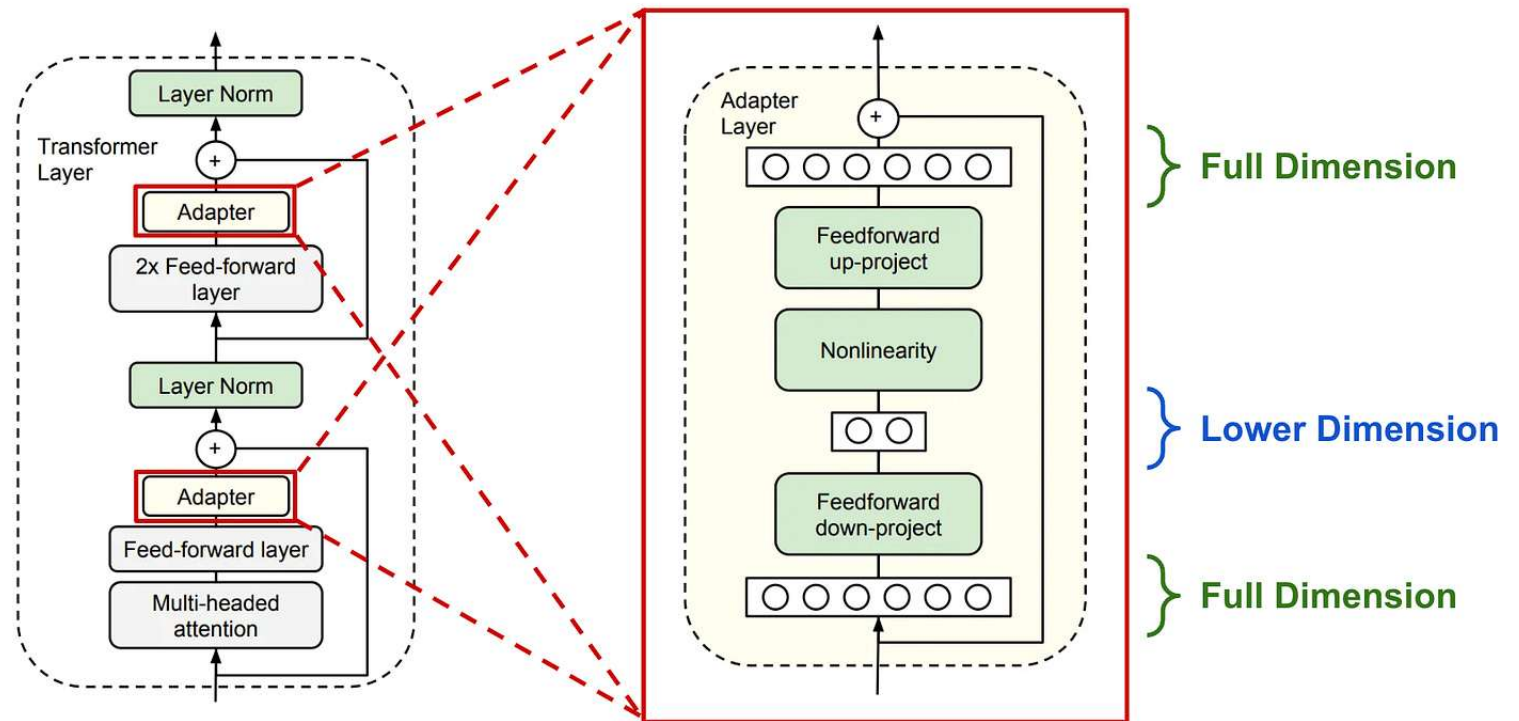
$$\text{Adapter}(h) = h + W_{up} \sigma(W_{down}h)$$

Where:

$$W_{down} \in \mathbb{R}^{d \times r}$$

$$W_{up} \in \mathbb{R}^{r \times d}$$

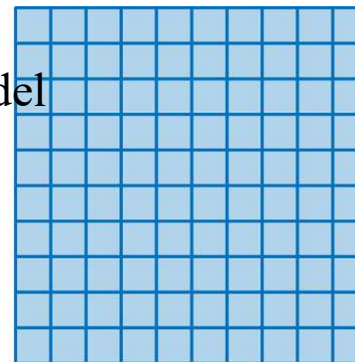
$$r \ll d$$



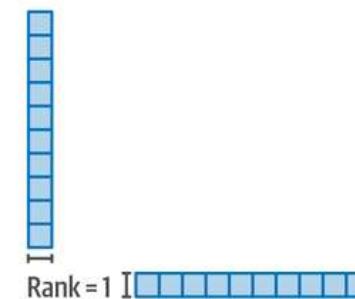
Low-Rank Adaptation (LoRA)

- **widely used** and effective technique (most popular)
- LoRA is a technique that (like adapters) **only requires updating a small set of parameters**.
- it creates a **small subset of the base model** to fine-tune instead of **adding layers to the model**
- **Assume the update ΔW can be approximated as low-rank:**
$$\Delta W = AB$$
- Trainable params per layer drop from $d_{\text{out}}d_{\text{in}}$ to $r(d_{\text{in}} + d_{\text{out}})$.
- You can **merge** $W \leftarrow W + \Delta W$ after training, so **no inference latency** penalty (unlike adapters).
- **Init:** commonly **one matrix is random**, the **other is zeros** so the model starts identical to base
- **Less prone to “catastrophic forgetting”** since the original model weights are kept the same.

Weight matrix
Full rank (10×10)
Total parameters: 100



Low-rank weight matrix (rank = 1)
Total parameters: 20



Low-rank weight matrix (rank = 2)
Total parameters: 40

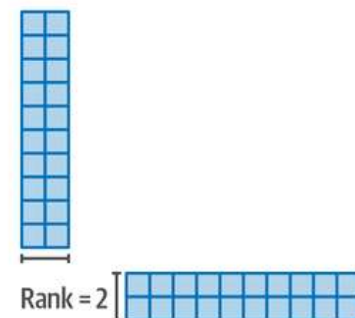


Diagram illustrating the LoRA adaptation process:

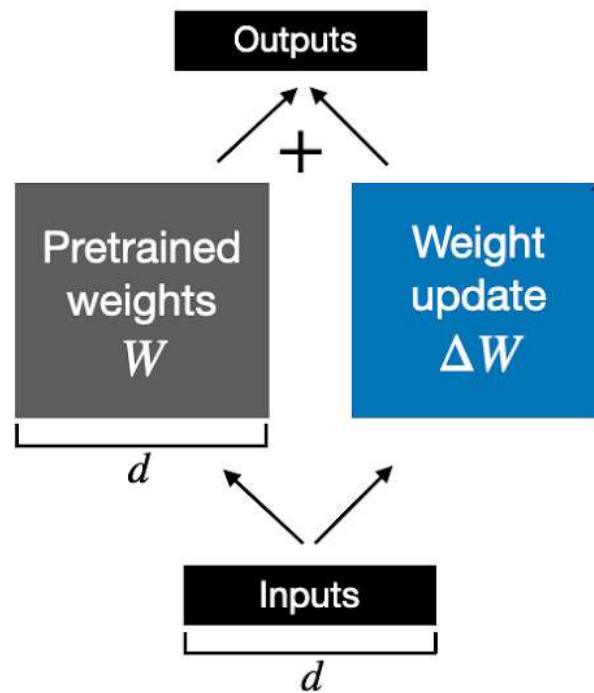
A large orange matrix of size $n \times n$ is added to a product of two smaller green matrices. The first green matrix is of size $r \times n$, and the second is of size $n \times r$. The product is labeled "low rank $r \ll n$ ".

$$W = W_0 + \underbrace{A}_{\text{frozen}} \underbrace{B}_{\text{trainable}}$$

LoRA

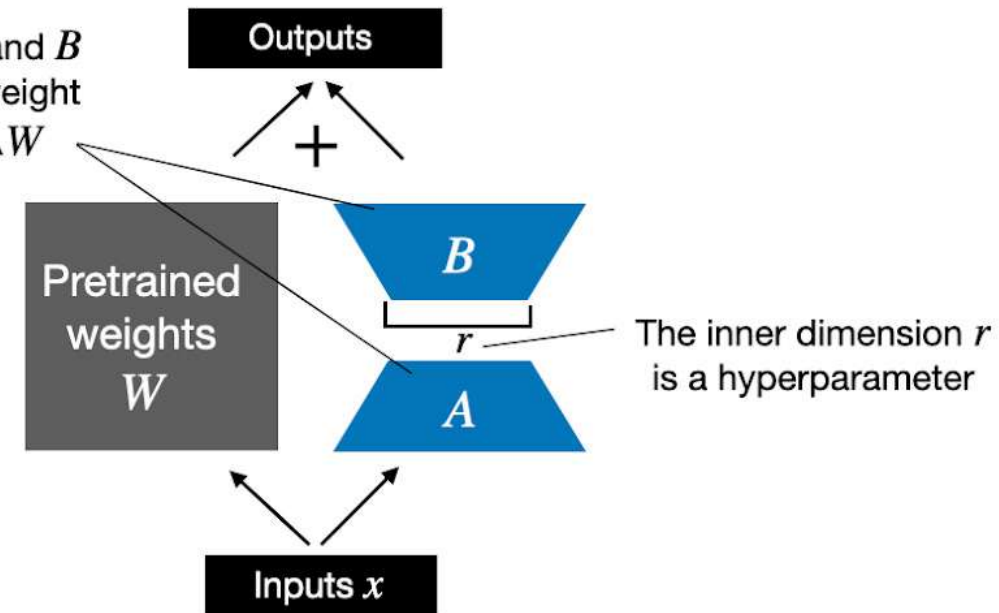
You can select which parts of the base model to fine-tune. For instance, we can only fine-tune the Query and Value weight matrices in each Transformer layer.

Weight update in regular finetuning



LoRA matrices A and B approximate the weight update matrix ΔW

Weight update in LoRA



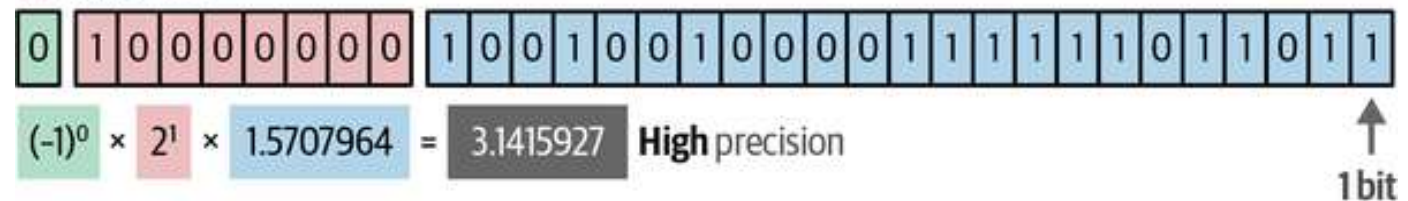
Papers like “**Intrinsic dimensionality explains the effectiveness of language model fine-tuning**” demonstrate that language models “have a very low intrinsic dimension.” This means that we can find small ranks that approximate even the massive matrices of an LLM.

A 175B model like GPT-3, for example, would have a **weight matrix of $12,288 \times 12,288$ inside each of its 96 Transformer blocks.** That’s 150 million parameters. If we can successfully adapt that **matrix into rank 8, that would only require two $12,288 \times 2$ matrices resulting in 197K parameters per block**

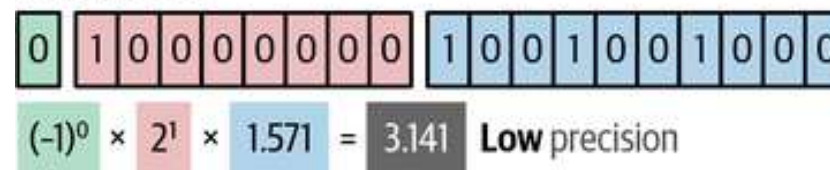
QLoRA – The Ingredients

- We can make LoRA even more efficient by **reducing the memory requirements of the model's original weights** before projecting them into smaller matrices.
- The weights of an LLM are numeric values with a given precision, which can be expressed by the number of bits like **float64 or float32**.
- If we **lower the amount of bits to represent a value**, we get a **less accurate** result. However, if we lower the number of bits we also **lower the memory** requirements of that model.
- There are **3 key ingredients** which helps us make QLoRA:
 - 4-Bit NormalFloat
 - Double Quantization
 - Paged Optimizer

Float 32-bit



Float 16-bit

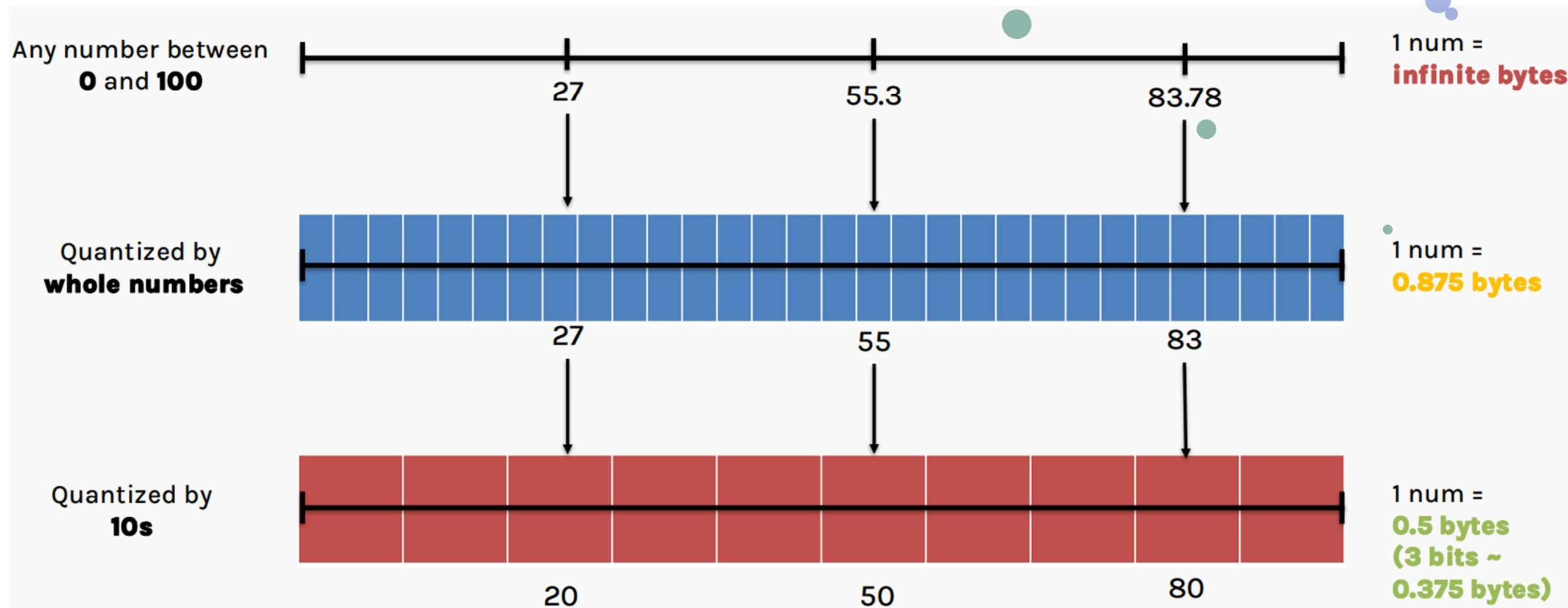


Quantization

To represent numbers from 0–100, you need: $\log_2(101) \approx 6.66$ bits
That's ≈ 7 bits, which is: $7/8 = 0.875$ bytes

Between 0 and 100 there are infinitely many values: 55.1, 55.01, 55.001, ... To store a truly arbitrary real number, you'd need infinite precision $\rightarrow$ conceptually infinite bytes

- Think of quantization as ‘splitting range into buckets’.



Example

- Let X be an array of $FP32$ values.

X^{FP32}	1.5	2.3	3.7	4.1	5.6	6.8	7.9	8.4	9.2	10.2
------------	-----	-----	-----	-----	-----	-----	-----	-----	-----	------

- What if we want to quantize from $FP32$ to $Int8$?

Int8 values range from -127 to 127

Array of values

$$X^{Int8} = \text{round} \left(\frac{127}{\text{absmax}(X^{FP32})} X^{FP32} \right)$$

The absolute maximum value in the array X^{FP32}

$$c^{FP32} = \frac{127}{\text{absmax}(X^{FP32})} = \frac{127}{10.2} = 12.4509$$

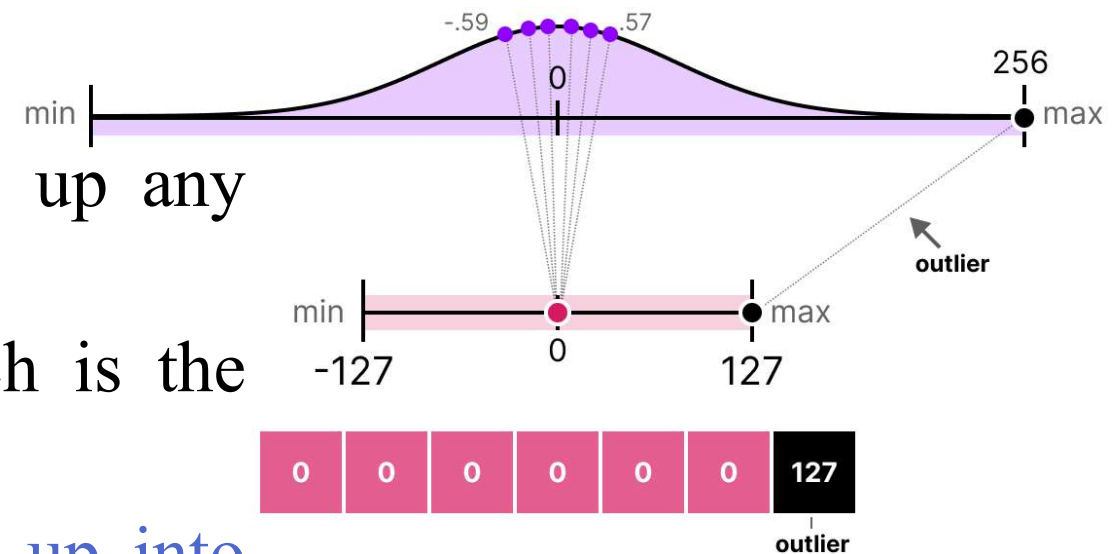
X^{Int8}	18	29	46	51	69	85	98	105	115	127
------------	----	----	----	----	----	----	----	-----	-----	-----

What if we want to dequantize and get back the original array, X^{FP32} ? (dequantization error)

$$X^{FP32} = \frac{X^{Int8}}{c^{FP32}}$$

Double Quantization

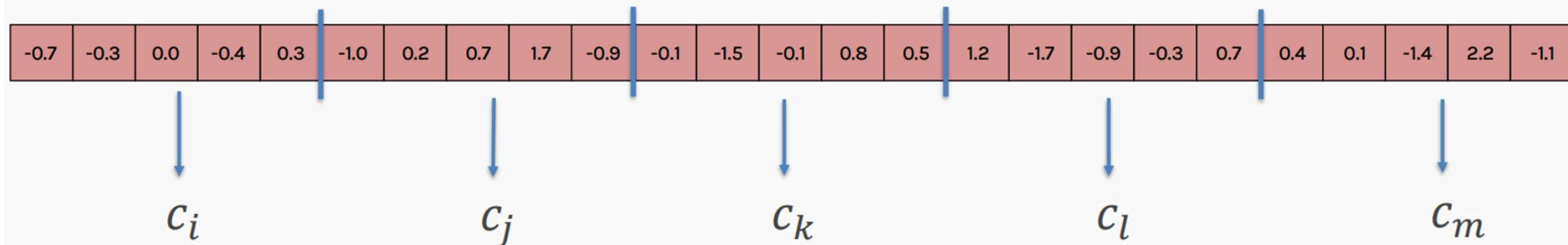
- **Problem:** This is unbounded and could take up any maximum value (an outlier!)
- The answer: **Block-wise Quantization**, which is the first step in Double Quantization!
- We flatten the matrix as follows & divide it up into different blocks.
- We calculate the quantization constants for each block.



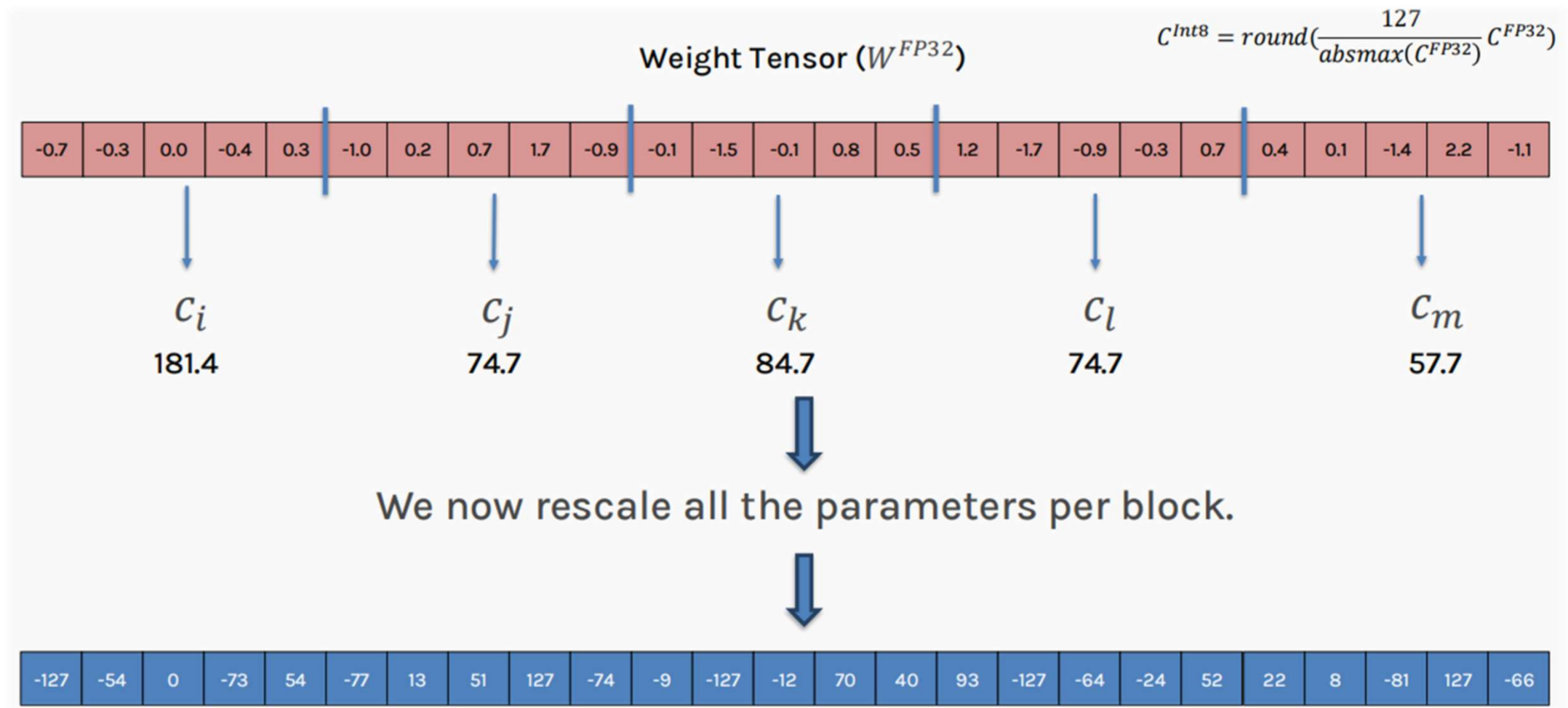
Weight Tensor (W^{FP32})

-0.7	-0.3	0.0	-0.4	0.3
-1.0	0.2	0.7	1.7	-0.9
-0.1	-1.5	-0.1	0.8	0.5
1.2	-1.7	-0.9	-0.3	0.7
0.4	0.1	-1.4	2.2	-1.1

Weight Tensor (W^{FP32})

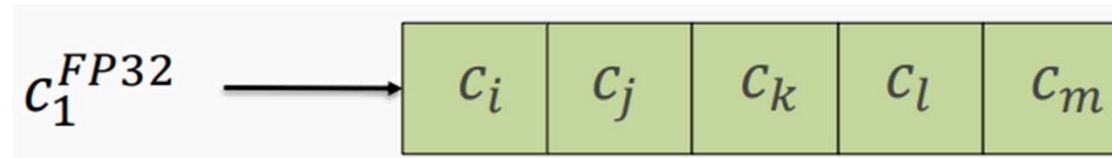


If there are any outliers in a block, they won't affect the quantisation in the other blocks.



-cont-

- We now have a new array:

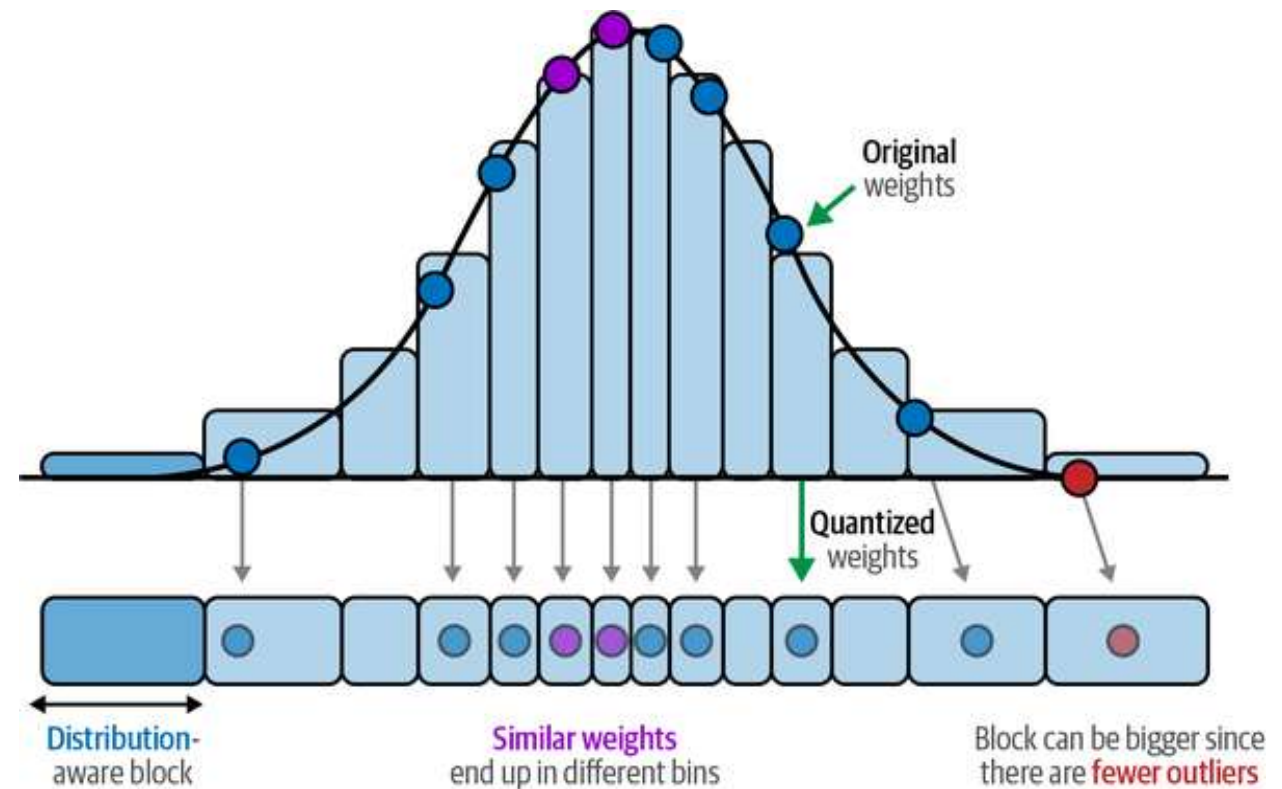


- Now, we repeat the same process of quantization for the **quantization constants**.

$$\left. \begin{aligned} c_1^{Int8} &= \text{round}\left(\frac{127}{\text{absmax}(c_1^{FP32})} c_1^{FP32}\right) \\ c_1^{Int8} &= \text{round}(c_2^{FP32} c_1^{FP32}) \end{aligned} \right\} \text{Double Quantization}$$

4-Bit NormalFloat

- 4-bit NormalFloat is a clever way to split the buckets.
- 4-bit means we have $2^4 = 16$ possible buckets for quantization.
- A nice property of neural networks is that their **values are generally normally distributed between -1 and 1**. This property allows us to bin the original weights to lower bits based on their relative density.
- The mapping between weights is more efficient as it takes into account the **relative frequency of weights**. This also reduces issues with outliers.



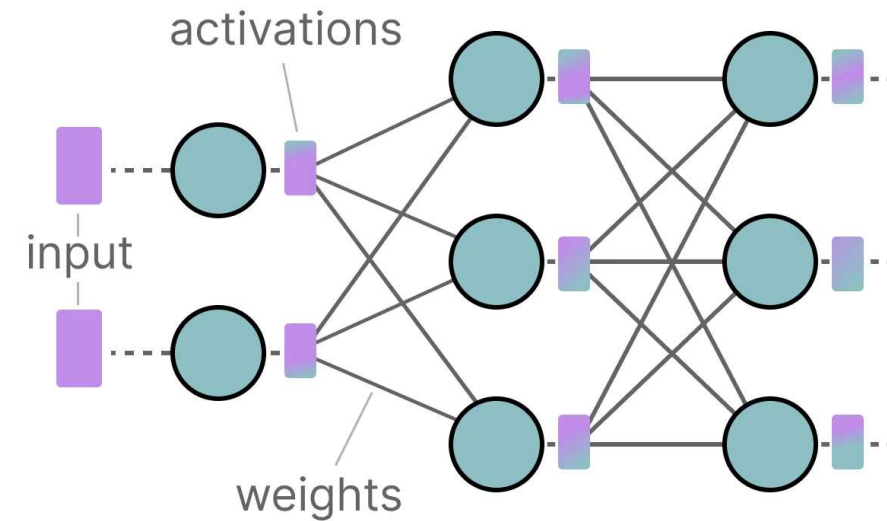
Paged Optimizer

- During training, a model must store **activations from the forward pass** so gradients can be computed in the backward pass.

- For large transformers:

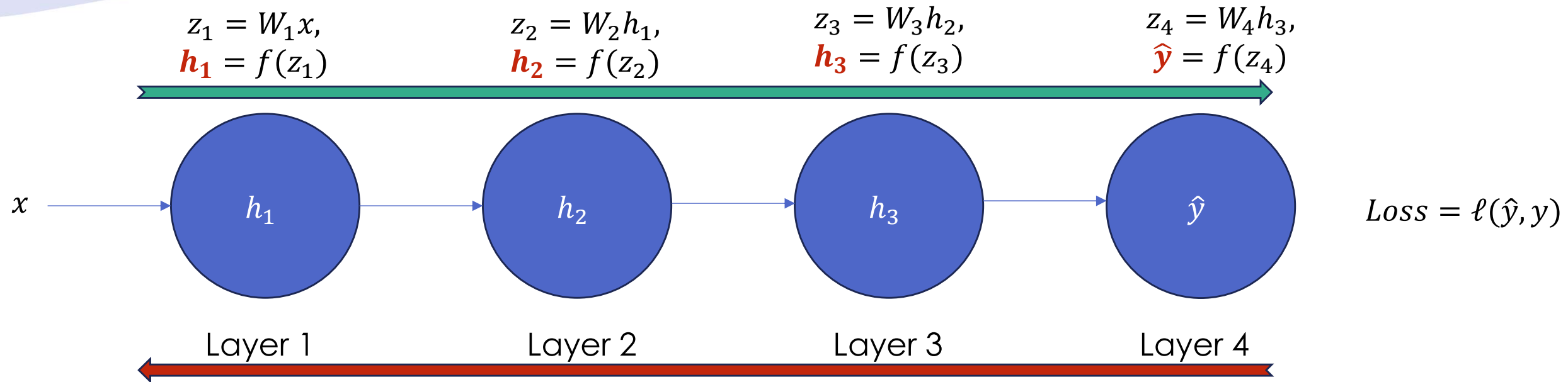
Activation memory $\gg$ model weights

- Memory scales with:
 - number of layers
 - sequence length
 - batch size
 - hidden dimension
- 🙌 This is often the reason you get **OOM during training even though inference fits**.



- **What gradient checkpointing does**
 - Instead of storing **all activations**, you **Save only a subset (“checkpoints”)**
 - **Recompute the missing activations on the fly during backward**
- In short:
 - **Save memory now, pay extra compute later**

Why we need to keep activations from forward pass



$$\frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial h_1} \odot f'(z_3)$$

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z_1} \cdot x$$

$$\frac{\partial L}{\partial x} = W_1^T \frac{\partial L}{\partial z_1}$$

Need x

You must have stored

$$\frac{\partial L}{\partial z_2} = \frac{\partial L}{\partial h_2} \odot f'(z_2)$$

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial z_2} \cdot h_1^T$$

$$\frac{\partial L}{\partial h_1} = W_2^T \frac{\partial L}{\partial z_2}$$

Need h_1

You must have stored

$$\frac{\partial L}{\partial z_3} = \frac{\partial L}{\partial h_3} \odot f'(z_3)$$

$$\frac{\partial L}{\partial W_3} = \frac{\partial L}{\partial z_3} \cdot h_2^T$$

$$\frac{\partial L}{\partial h_2} = W_3^T \frac{\partial L}{\partial z_3}$$

Need h_2

You must have stored

$$\frac{\partial L}{\partial z_4} = \frac{\partial Loss}{\partial \hat{y}}$$

$$\frac{\partial L}{\partial W_4} = \frac{\partial L}{\partial z_4} \cdot h_3^T$$

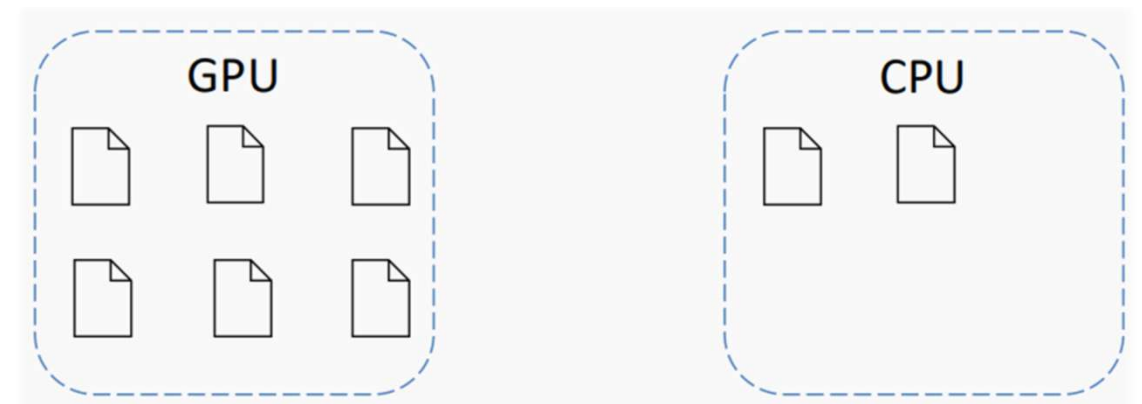
$$\frac{\partial L}{\partial h_3} = W_4^T \frac{\partial L}{\partial z_4}$$

Need h_3

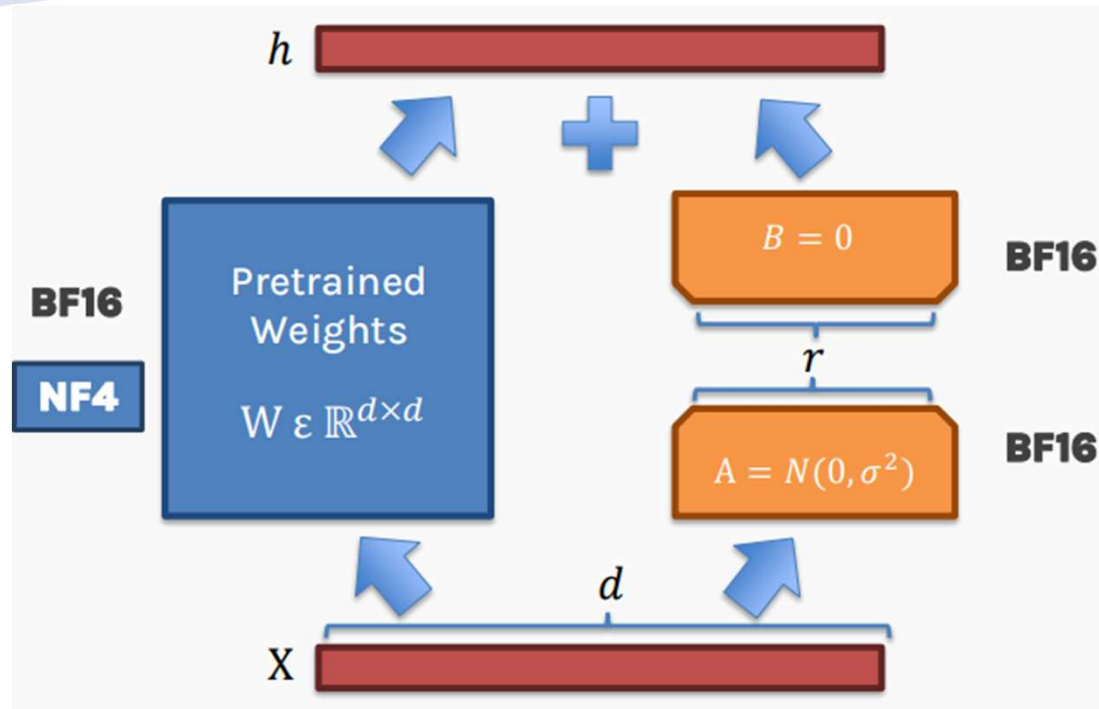
You must have stored

Gradient checkpointing

- To avoid hitting OOM, We only store in memory what is needed at the moment.
- We keep discarding activations that have already been used to calculate the next dependent hidden state's activation.
- During backpropagation, we must recompute all the discarded activations. To manage this, we introduce checkpoints in the middle.
- Checkpoints are usually placed at every $\sqrt{n}$ layer, considering we have a n -layer neural network. So, now when we re-compute the activations for backward pass, we don't have to start from the beginning!
- This allows us to mitigate the OOM (Out of memory) error to some extent, but it doesn't get rid of it! We still see some memory spikes especially when we pass in long sequences in the batch.
- **Paged Optimizer - Looping in your CPU**
- It does automatic page-to-page transfers between CPU and GPU
- Avoids the gradient checkpointing memory spikes that occur when processing a mini batch with a long sequence length
- Now that the GPU has space, when a page moved to CPU is required, we move it back to GPU for computation.

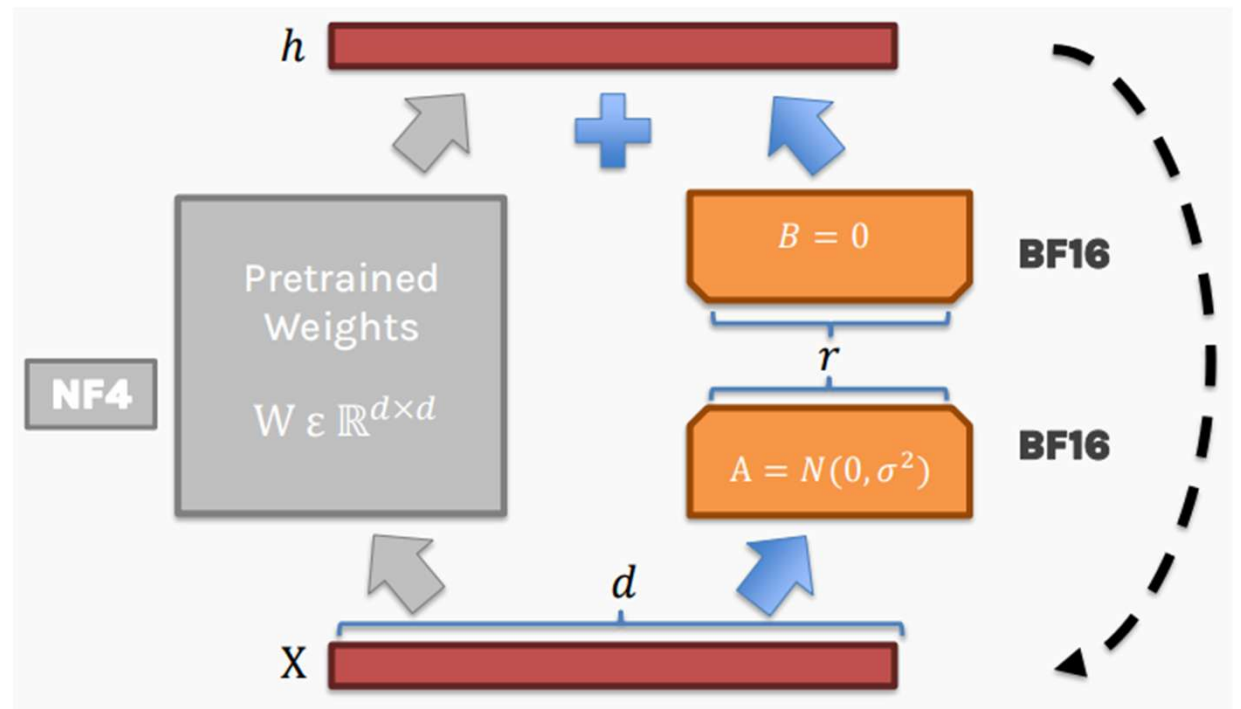


QLoRA



- **Forward pass**

- During the forward pass, we first **dequantize the W weights from NF4 to BF16** for computation
- We then **use the BF16 values of W , A and B to perform the required calculations.**
- The **BF16 values of W is then deleted** to save on storage!

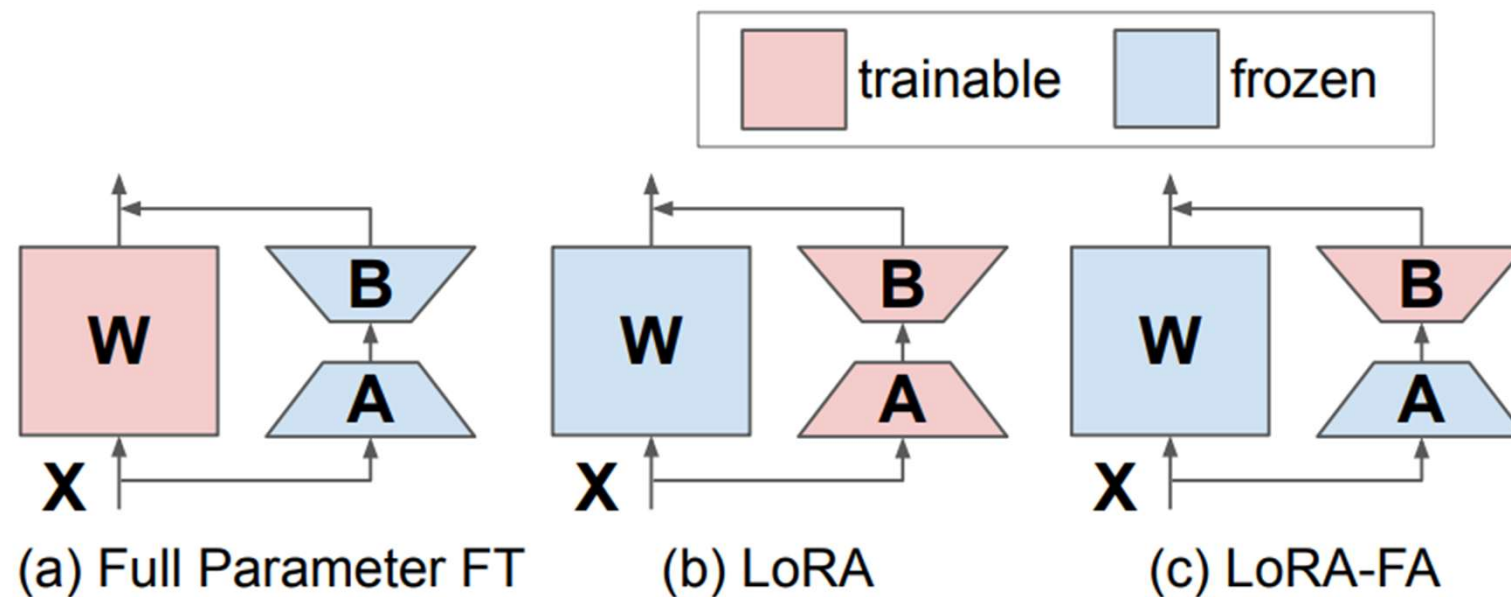


- **Backward pass**

- As in LoRA, we **keep W weights frozen** and allow the gradients to only flow through the adapters.
- We then repeat the cycle of forward and backward passes till a minima is reached.

LoRA-FA (LoRA with Frozen-A)

- During inference, activations are created as a **product of the input and the weights, which similarly can be quite large.**
- LoRA reduces trainable weights, but **activation memory can still be big** because standard backprop for LoRA needs the *layer input activation* to compute gradients for both A and B
- The **matrix A is frozen after initialization** and hence serves as a random projection. Instead of adding new vectors, **matrix B is trained though**, after being initialized with zeros



Why activation memory is still big in standard LoRA?

- LoRA branch computes BAx . Often they write it as:

$$z = Ax \text{ (size } r\text{)}$$

add Bz to the output

- Even though LoRA reduces trainable parameters, backprop still needs saved activations.
- In standard LoRA, you train both A and B . Gradients look like:
- For B : needs $z = Ax$

$$\nabla_B \propto g z^\top$$

- For A : needs the original layer input activation x

$$\nabla_A \propto (B^\top g) x^\top$$

- Here $g = \frac{\partial \mathcal{L}}{\partial y}$ is the upstream gradient.
- Key point:** to compute ∇_A , you must keep x around (or recompute it). But x is big: shape (batch $\times$ seq $\times d_{in}$).

- LoRA-FA changes one thing:

- Initialize A randomly. **Freeze A forever**. Initialize $B = 0$ and **train B** . So A becomes a **fixed random projection** into an r -dim space:

$$z = Ax$$

- With A frozen, you no longer need ∇_A . That removes the need to store big x for this layer's LoRA gradients.
- For training B , you only need $z = Ax$, which is much smaller:

x has width d_{in}

z has width r

- So the saved activation for LoRA can shrink by roughly:

$$\text{memory reduction factor} \approx \frac{d_{in}}{r}$$

- Example: if $d_{in} = 4096$ and $r = 16$, that's **$\sim 256\times$ smaller** for that part.

Extensions

LoRA+

using the same learning rate for A and B does not allow efficient feature learning. suboptimality of LoRA can be corrected simply by setting **different learning rates** for the LoRA adapter matrices A and B with a well-chosen fixed ratio

VeRA

Instead of training the matrices A and B, which is the core idea of LoRA in the first place, they **initialize these matrices with shared random weights** (i.e. all the matrices A and B in all the layers have the same weights) and **add two new vectors d and b**. Only these vectors d and b are **trained** in the following.

	LoRA	LoRA+
Parameterization	Pretrained Weights $W \in \mathbb{R}^{n \times n}$ + B × A	
Training	$A \leftarrow A - \eta \times G_A$ $B \leftarrow B - \eta \times G_B$	$A \leftarrow A - \eta \times G_A$ $B \leftarrow B - \lambda \eta \times G_B$ $\lambda \gg 1$

